SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 4 OF 10

REST APIs, Auth, JWT & Security

11 Questions | OAuth 2.0 | JWT Internals | CSRF/XSS/CORS | BFF Pattern | Rate Limiting

REST APIs, Auth, JWT & Security

Q1: What makes a good REST API? Core principles and common violations.

REST (Representational State Transfer) is an architectural style, not a protocol. Good REST APIs are predictable, consistent, and designed around resources — not actions.

Resource-oriented URLs: URLs identify resources (nouns), not operations (verbs). /users/42 is correct. /getUser?id=42 is not REST — that's RPC over HTTP. Actions map to HTTP methods: GET=read, POST=create, PUT=replace, PATCH=update, DELETE=remove.

Statelessness: Every request must contain all information needed to process it. The server holds no client session state between requests. Auth tokens, pagination cursors, and filters go in the request — not server-side sessions.

Consistent response structure: Always return the same shape for the same endpoint regardless of outcome. Errors should have a consistent structure: {error, message, code, details}. Never return HTML error pages from a JSON API.

HTTP status codes used correctly: 200 OK, 201 Created (POST that creates), 204 No Content (DELETE success), 400 Bad Request (client error), 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but not allowed), 404 Not Found, 409 Conflict, 422 Unprocessable Entity (validation failed), 429 Too Many Requests, 500 Internal Server Error.

```
# Bad REST — RPC style
POST /createUser
POST /deleteUser
GET  /getUserById?id=42

# Good REST — resource style
POST   /users             -> 201 + {id, name, email}
GET    /users             -> 200 + [{...}, {...}]
GET    /users/42          -> 200 + {id, name, email}
PATCH  /users/42          -> 200 + updated user
DELETE /users/42          -> 204 No Content

# Nested resources
GET  /users/42/orders     -> orders belonging to user 42
POST /users/42/orders     -> create order for user 42
```

HATEOAS: Hypermedia As The Engine Of Application State — responses include links to related actions. Pure HATEOAS is rarely implemented in practice but its spirit (discoverable APIs, embedding related resource IDs) is worth following.

Q2: PUT vs PATCH — what's the actual difference and when does it matter?

The distinction is not just semantic — it changes how clients and servers must behave and has real implications for concurrency and partial updates.

PUT: Full replacement. The client sends the complete representation of the resource. Missing fields are deleted or set to null. PUT must be idempotent — calling it N times has the same effect as calling it once.

PATCH: Partial update. The client sends only the fields to change. The server applies the diff to the current state. PATCH is NOT required to be idempotent (though it often is in practice).

```
# Current user: {id: 1, name: 'Gautham', email: 'g@test.com', role: 'admin'}

# PUT — must send EVERYTHING or fields get lost
PUT /users/1
{"name": "Gautham G", "email": "g@test.com", "role": "admin"}
```

```
# role must be included — omitting it would delete/null it

# PATCH — send only what changes
PATCH /users/1
{"name": "Gautham G"}
# email and role untouched — only name updated

# JSON Patch (RFC 6902) — explicit patch operations
PATCH /users/1
Content-Type: application/json-patch+json
[
  {"op": "replace", "path": "/name", "value": "Gautham G"},
  {"op": "remove",  "path": "/website"}
]
```

Idempotency matters for retries: If a network timeout occurs, the client can safely retry PUT — the result is the same. Retrying a non-idempotent PATCH (like an increment operation) would apply the change twice. Design PATCH payloads to be idempotent (set to value, not add/subtract) when possible.

Q3: API versioning strategies — tradeoffs of each approach.

API versioning is about managing breaking changes without forcing all clients to upgrade simultaneously. There is no universally correct approach — each has real engineering tradeoffs.

URL versioning (/v1/users): Most common. Explicit, visible, easy to route in API gateway. Downside: resource duplication in URLs, violates REST purity (the URL should identify a resource, not a version of a resource).

Header versioning (Accept: application/vnd.myapi.v2+json): REST-pure. URL stays clean. Downside: cannot be tested directly in browser, harder to cache (cache key must include the header), harder for developers to discover.

Query param (?version=2): Easy to add for testing. Downside: easy to forget, can be stripped by proxies, pollutes query strings.

Sunset headers: When deprecating old versions, return Sunset: Sat, 01 Jan 2026 00:00:00 GMT and Deprecation: true headers. Clients can monitor these and alert teams before the version is removed.

```
# URL versioning — most pragmatic for public APIs
GET /v1/users/42      # old version — kept for backward compat
GET /v2/users/42      # new version — different response shape

# FastAPI router versioning
v1_router = APIRouter(prefix='/v1')
v2_router = APIRouter(prefix='/v2')
v1_router.include_router(users_v1.router)
v2_router.include_router(users_v2.router)
app.include_router(v1_router)
app.include_router(v2_router)
```

Q4: Pagination — offset vs cursor vs keyset. Which and when?

Pagination is not just a UX concern — it directly impacts DB query performance and API consistency under concurrent writes.

Offset pagination (page=2&size=20): Simple to implement. SELECT ... LIMIT 20 OFFSET 40. Breaks under concurrent inserts (items shift, causing skips or duplicates between pages). Performance degrades on large offsets — DB must count and skip rows.

Cursor pagination: Client sends an opaque cursor (usually an encoded ID or timestamp). Server fetches rows WHERE id > cursor LIMIT 20. Stable under writes. O(1) regardless of depth. Cannot jump to arbitrary pages. Best for infinite scroll, feeds, event streams.

Keyset pagination: Like cursor but uses the actual indexed column value. WHERE (created_at, id) > ('2024-01-01', 42) — exploits composite index. Very fast. Requires a stable sort key.

```
# Cursor pagination response
{
  "data": [...],
  "pagination": {
    "next_cursor": "eyJpZCI6IDQyfQ==",   // base64(json({id: 42}))
    "has_more": true
  }
}

# Server decodes cursor and queries
cursor_data = decode_cursor(cursor)   # {id: 42}
query = 'SELECT * FROM users WHERE id > $1 ORDER BY id LIMIT $2'
rows = await db.fetch(query, cursor_data['id'], page_size + 1)
# fetch page_size+1 to determine has_more without extra COUNT query
has_more = len(rows) > page_size
data = rows[:page_size]
```

Q5: Rate limiting — strategies, implementation, and headers.

Rate limiting protects your API from abuse, prevents runaway clients, and ensures fair resource distribution. The algorithm choice affects how smoothly limits are enforced.

Fixed window: Count requests per time window (e.g., 100 req/minute). Simple. Vulnerable to burst at window boundaries — a client can make 100 requests at 00:59 and 100 more at 01:01 for 200 in 2 seconds.

Sliding window: Track exact timestamps of requests. More accurate than fixed window. More memory intensive. Eliminates the boundary burst problem.

Token bucket: Tokens accumulate at a fixed rate (up to a max). Each request consumes a token. Allows controlled bursting up to the bucket size. Redis-based with atomic operations. Most flexible for real-world traffic patterns.

Leaky bucket: Requests processed at a constant rate regardless of arrival rate. Excess requests queue or are dropped. Smoothest output but adds latency for bursty clients.

```
# Redis token bucket implementation (atomic Lua script)
RATE_LIMIT_SCRIPT = """
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])
local data = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens = tonumber(data[1]) or capacity
local last = tonumber(data[2]) or now
local delta = math.max(0, now - last)
tokens = math.min(capacity, tokens + delta * refill_rate)
if tokens >= requested then
  tokens = tokens - requested
  redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
  return 1
end
return 0
"""

# Standard rate limit response headers
X-RateLimit-Limit: 100
```

```
X-RateLimit-Remaining: 42
X-RateLimit-Reset: 1704067200
Retry-After: 30          # included on 429 response
```

Q6: JWT structure, how it works, and what are its real risks?

JWT (JSON Web Token) is a compact, self-contained token format for transmitting claims between parties. Understanding its structure and failure modes is essential for building secure APIs.

Structure: Three base64url-encoded parts separated by dots: Header.Payload.Signature. Header: algorithm and token type. Payload: claims (sub, exp, iat, custom fields). Signature: HMAC or RSA signature of header+payload.

```
# Header (decoded)
{"alg": "HS256", "typ": "JWT"}

# Payload (decoded) — NEVER store sensitive data here
{
  "sub": "user_42",         // subject (user ID)
  "iat": 1704067200,          // issued at
  "exp": 1704070800,          // expires at (1 hour later)
  "roles": ["admin"],
  "jti": "uuid-for-this-token"  // JWT ID — enables revocation
}

# Signature
HMAC-SHA256(base64(header) + '.' + base64(payload), secret_key)

# Verification: recalculate signature, compare, check exp
# Payload is readable by anyone — base64 is encoding, NOT encryption
```

Why JWT is stateless: The server does not store any session state. Every request carries the token; the server validates the signature and expiry. This makes horizontal scaling trivial — any server instance can validate any token without a shared session store.

Risk: algorithm confusion (alg:none): Historically, servers that trusted the alg header could be fooled by setting alg to 'none' — bypassing signature verification. Always hardcode the expected algorithm on the server side. Never accept alg from the token.

Risk: long expiry: JWTs cannot be revoked (stateless). A stolen token is valid until expiry. Keep access tokens short-lived (5-15 minutes). Use refresh tokens for long sessions.

Risk: sensitive data in payload: The payload is base64 encoded — anyone can read it. Never put passwords, PII beyond user ID, or internal system details in the payload. If you need confidentiality, use JWE (encrypted JWT).

Where to store JWT: Access token: in-memory (JavaScript variable). NOT localStorage (XSS vulnerable). NOT sessionStorage. Refresh token: HttpOnly, Secure, SameSite=Strict cookie. HttpOnly prevents JavaScript access entirely.

Q7: Refresh token rotation — how it works and why it prevents replay attacks.

Refresh token rotation is a security pattern where each use of a refresh token invalidates the old one and issues a new one. This limits the window of opportunity if a refresh token is stolen.

```
# Flow:
1. Login -> server issues: access_token (15min) + refresh_token (30 days)
2. Store refresh_token in HttpOnly cookie
3. Store access_token in memory (JS variable, lost on tab close)
```

```
4. access_token expires
5. Client sends POST /auth/refresh with refresh_token cookie
6. Server:
   a. Validates refresh_token signature and expiry
   b. Checks refresh_token is NOT in revoked set
   c. Marks OLD refresh_token as used/revoked (in DB/Redis)
   d. Issues NEW access_token + NEW refresh_token
   e. Returns new tokens

# Replay attack scenario:
# Attacker steals refresh_token at time T
# Legitimate user uses it at T+1 -> gets new tokens, old one revoked
# Attacker tries to use stolen token at T+2 -> REJECTED (already used)
# Server detects reuse -> revoke ALL tokens for this user (breach detected)
```

Reuse detection: If a refresh token that was already rotated is presented again, it means a token was stolen and used. The correct response is to immediately invalidate all refresh tokens for that user and force re-login. This is breach detection built into the auth flow.

Storage: Refresh tokens must be stored server-side (DB or Redis) to support rotation and revocation. This adds some statefulness but only to refresh tokens — access token validation remains stateless.

Q8: CSRF, XSS, CORS — explain each and how you prevent them.

These are the three most common web security attack classes. A senior engineer must know the mechanism of each — not just 'add a header' but why the header works.

XSS (Cross-Site Scripting): Attacker injects malicious JavaScript into your page. If access tokens are in localStorage, XSS can steal them. Prevention: Content-Security-Policy header, sanitize user input, use HttpOnly cookies for sensitive tokens (JS cannot access them).

CSRF (Cross-Site Request Forgery): Attacker tricks the victim's browser into making an authenticated request to your API from a malicious site. The browser automatically sends cookies — including your session cookie. Prevention: SameSite=Strict/Lax on cookies (browser won't send them on cross-site requests), CSRF tokens in forms, check Origin/Referer headers.

CORS (Cross-Origin Resource Sharing): Browser security policy that blocks cross-origin requests by default. Your API must explicitly allow origins via CORS headers. CORS is enforced by the browser — it does NOT protect against server-to-server calls or curl.

```
# CORS headers — what the server returns
Access-Control-Allow-Origin: https://myapp.com   # NOT * for credentialed requests
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true           # required for cookies

# FastAPI CORS setup
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=['https://myapp.com'],   # explicit, never '*' with credentials
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

# SameSite cookie for CSRF prevention
Set-Cookie: refresh_token=...; HttpOnly; Secure; SameSite=Strict; Path=/auth
```

SameSite=Strict vs Lax: Strict: cookie never sent on cross-site requests (even top-level navigation). Lax: cookie sent on top-level GET navigations but not on POST/PUT/DELETE cross-site requests. For refresh tokens, use Strict. For session cookies on SPAs, Lax is often acceptable.

Q9: OAuth 2.0 — flows, roles, and when to use which grant type.

OAuth 2.0 is an authorization framework — it lets users grant third-party apps access to their resources without sharing credentials. Understanding the flows is essential for both building OAuth servers and integrating with providers (Google, GitHub).

Roles: Resource Owner (user), Client (your app), Authorization Server (issues tokens — e.g., Google's auth), Resource Server (API that accepts tokens — e.g., Google Drive API).

Authorization Code + PKCE: The standard flow for web and mobile apps. User is redirected to auth server, logs in, consents, auth server redirects back with a code. Client exchanges code for tokens. PKCE (Proof Key for Code Exchange) prevents code interception attacks. Use this for anything where a user is involved.

Client Credentials: Machine-to-machine. No user involved. Client sends client_id + client_secret, gets access token. Used for microservice-to-microservice auth, background jobs, CI/CD systems.

Implicit flow (deprecated): Tokens returned directly in URL fragment. Vulnerable to token leakage. Never use — replaced by Authorization Code + PKCE.

```
# Authorization Code + PKCE flow
1. Client generates: code_verifier (random), code_challenge = SHA256(code_verifier)

2. Redirect user to:
   GET /oauth/authorize?
     response_type=code
     &client_id=my_app
     &redirect_uri=https://myapp.com/callback
     &scope=read:profile email
     &state=random_csrf_value
     &code_challenge=BASE64URL(SHA256(verifier))
     &code_challenge_method=S256

3. User authenticates and consents
4. Auth server redirects: /callback?code=AUTH_CODE&state=random_csrf_value
5. Client verifies state (CSRF check)
6. Client exchanges: POST /oauth/token
   {grant_type: authorization_code, code: AUTH_CODE, code_verifier: VERIFIER}
7. Auth server returns: {access_token, refresh_token, expires_in}
```

Authentication vs Authorization: OAuth is for AUTHORIZATION — granting access to resources. OpenID Connect (OIDC) is the authentication layer on top of OAuth. OIDC adds an ID token (JWT with user info) alongside the access token. 'Login with Google' is OIDC, not plain OAuth.

Q10: How do cookies work — first-party vs third-party, and the BFF cookie pattern?

Cookies are key-value pairs stored by the browser and automatically sent with every matching request. Your resume mentions building a BFF reverse proxy to convert third-party to first-party cookies — this is worth explaining in detail.

First-party cookies: Set by the domain the user is directly visiting. If the user is on myapp.com and the server at myapp.com sets a cookie, it's first-party. Sent on all requests to myapp.com. Not blocked by browsers.

Third-party cookies: Set by a domain different from the page the user is on. Used for cross-site tracking (analytics, ads). Safari ITP and Chrome have blocked/restricted them. This breaks many tracking and auth flows that relied on cross-site cookies.

BFF pattern (Backend for Frontend): The BFF sits on the same domain as the frontend (first-party). Instead of the SPA calling a third-party API directly (which would set third-party cookies that get blocked), the SPA calls the BFF. The BFF proxies the request to the real API and can rewrite Set-Cookie headers so cookies appear to come from the first-party domain.

```
# Problem: SPA on myapp.com, API on api.vendor.com
```

```
# Vendor sets: Set-Cookie: session=abc; Domain=api.vendor.com
# Browser blocks this as third-party on Safari/Chrome

# Solution: BFF proxy on same domain
# SPA calls: myapp.com/bff/vendor/resource
# BFF (on myapp.com) forwards to api.vendor.com
# Vendor response has Set-Cookie for api.vendor.com
# BFF strips that cookie, stores the value, sets its own:
# Set-Cookie: session=abc; Domain=myapp.com; HttpOnly; Secure
# Now it's first-party — browser accepts it

# Go implementation sketch
func bffProxy(w http.ResponseWriter, r *http.Request) {
    resp := forwardToVendor(r)
    // Rewrite cookies to first-party
    for _, cookie := range resp.Cookies() {
        cookie.Domain = 'myapp.com'
        http.SetCookie(w, cookie)
    }
    copyBody(w, resp)
}
```

Cookie attributes: Domain: which domains receive the cookie. Path: which paths. Expires/Max-Age: persistence. Secure: HTTPS only. HttpOnly: no JavaScript access. SameSite: cross-site sending rules. Always set Secure + HttpOnly + SameSite on auth cookies.

Q11: API Gateway vs Reverse Proxy vs Load Balancer — what each does and when you need each.

These terms are often conflated but they operate at different layers and serve different purposes. In production microservice stacks, you typically have all three.

Reverse Proxy (nginx, Caddy): Sits in front of your services. Terminates SSL, forwards requests to the right backend, handles static files, connection pooling, and basic load balancing. Single service or simple multi-service routing. Layer 7 HTTP routing.

Load Balancer (HAProxy, AWS ALB): Distributes traffic across multiple instances of the same service. Algorithms: round-robin, least connections, IP hash (sticky sessions). Can operate at Layer 4 (TCP) or Layer 7 (HTTP). Health checks remove unhealthy instances.

API Gateway (Kong, AWS API Gateway, Traefik): A specialized reverse proxy for microservices. Adds: auth/JWT validation, rate limiting, request transformation, API key management, analytics, routing to multiple different backend services, circuit breaking, and canary deployments — all without changing your service code.

```
# Typical production stack:

Internet
    -> CDN (CloudFront) — static assets, edge caching
    -> Load Balancer (ALB) — distributes to API Gateway instances
    -> API Gateway (Kong/Traefik)
         -> Auth validation (JWT check on every request)
         -> Rate limiting (per API key / per user)
         -> Routes:
              /api/users/*  -> User Service (3 instances)
              /api/orders/* -> Order Service (2 instances)
              /api/search/* -> Search Service (5 instances)
         -> Each service has its own internal load balancer
```

When you don't need an API Gateway: Monolith or single-service APIs. Simple internal microservices with a service mesh (Istio handles auth + observability). API Gateway adds operational complexity — earn it before adopting it.
